

System Design:

1. Requirement Clarification & Assumptions

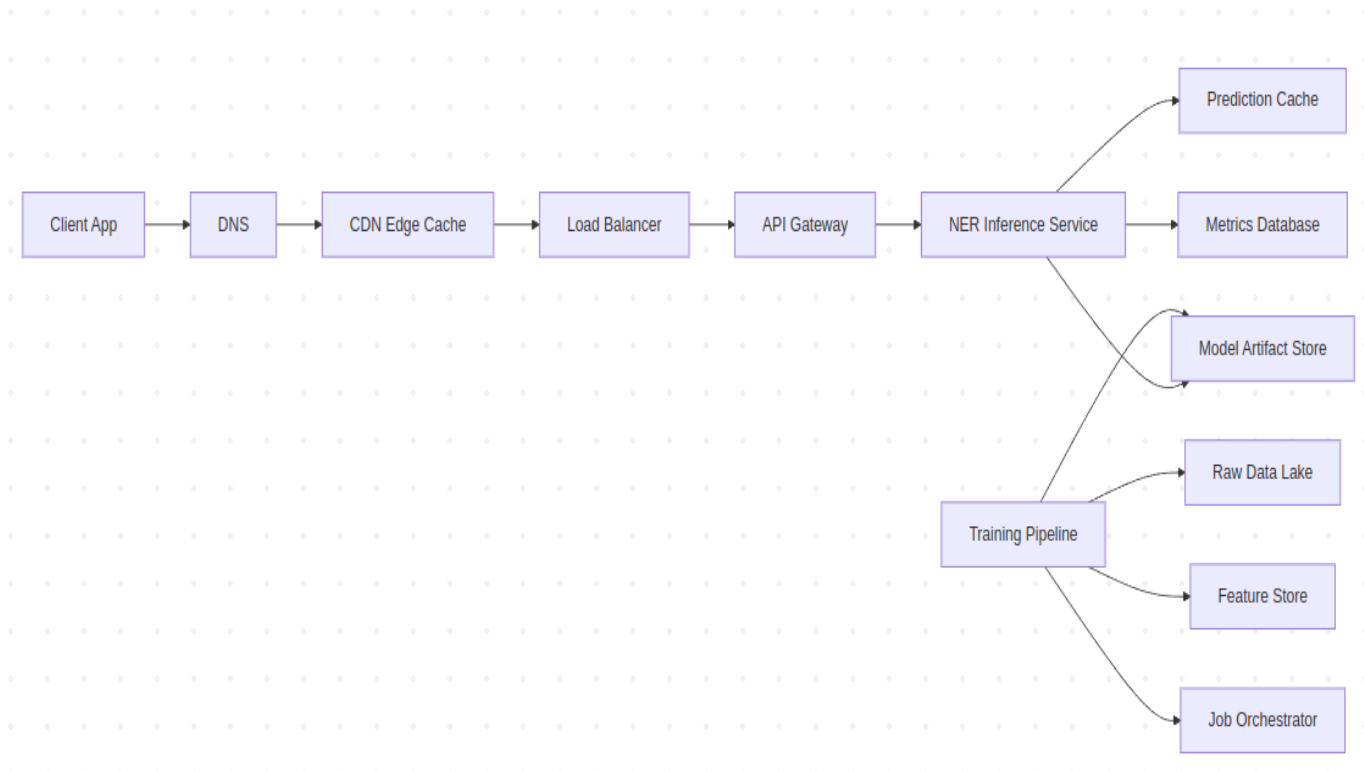
Assumptions

- Peak load: 200 QPS, average: 50 QPS
- Per-request latency target: <200 ms
- Inference on GPU instances (e.g. NVIDIA T4)
- Model retraining daily on new annotated data
- 99.9 % service availability across a single AWS region
- Budget allows moderate cloud GPU and managed services

2. Back-of-the-Envelope Estimation

Metric		Estimate
Peak QPS	200	
Avg payload size	~300 tokens/request	
Latency per inference	~100 ms (GPU)	
Throughput per GPU node	~250 QPS	
Storage for logs	$200 \text{ QPS} \times 86400 \text{ s} \times 365 \text{ d} \approx 5 \text{ billion logs/year} \rightarrow \sim 50 \text{ GB (JSON)}$	
Model artifacts size	~500 MB per version	
Training data stored	$\sim 20 \text{ K sentences} \times 128 \text{ tokens} \times 4 \text{ bytes} \approx 10 \text{ MB}$	

3. High-Level Architecture



- **Client App**
 - Sends text for tagging; can be web, mobile, or backend.
 - Impact: workload origin; influences rate limiting and CORS.
- **DNS**
 - Resolves domain to edge/CDN endpoints.
 - Improves availability and global reach.
- **CDN Edge Cache**
 - Caches frequent static responses or model metadata.
 - Reduces load on origin and lowers latency.
- **Load Balancer**
 - Distributes API calls across inference nodes.
 - Ensures horizontal scalability and fault-tolerance.

- **API Gateway**
 - Handles authentication, rate limiting, routing to services.
 - Centralizes cross-cutting concerns; can throttle traffic.
- **NER Inference Service**
 - Loads BERT+CRF model in GPU memory; serves prediction requests.
 - Core latency component; uses batch queue for throughput.
- **Cache**
 - Stores recent predictions keyed by text hash.
 - Reduces repeated inference on identical inputs.
- **Model Artifact Store**
 - Stores serialized models (TorchScript/ONNX) with versions.
 - Enables safe rollbacks and A/B testing.
- **Metrics Database**
 - Time-series store of latency, error rates, throughput.
 - Drives alerts when SLA thresholds are breached.
- **Data Lake & Feature Store**
 - Raw logs and engineered features for retraining.
 - Ensures reproducible data pipelines.
- **Job Orchestrator**
 - Schedules daily training, evaluation, and deployment.
 - Coordinates data movement and model promotion.

Protocols & Scaling

- **Communication:** HTTP 2 over TLS between client and gateway; gRPC between gateway and inference service for efficiency.

- **Scaling:** Horizontal autoscaling of inference pods based on GPU utilization and queue length; data stored in S3-compatible lake; metadata in managed PostgreSQL.
 - **Partitioning:** Shard metrics and logs by day; noSQL cache uses consistent hashing; model store versioned by key.
 - **Replication:** API Gateway and Load Balancer in at least 2 AZs; inference pods replicated; data lake with cross-region replication for DR.
-

4. Component Analysis & Trade-Offs

API Gateway

What it is: Central ingress point for all requests, enforcing auth and rate limits.

Why chosen: Simplifies handling cross-cutting concerns (security, throttling).

Trade-Offs

- **Benefit:** Single configurable boundary; metrics in one place.
- **Drawback:** Potential single point of misconfiguration; adds ~5 ms latency.

NER Inference Service

What it is: GPU-backed microservice loading BERT+CRF for token tagging.

Why chosen: GPUs accelerate transformer inference to meet sub-200 ms SLA.

Trade-Offs

- **Benefit:** High throughput (~250 QPS) and low latency (~100 ms).
- **Drawback:** Higher cost; must manage GPU drivers and utilization.

Prediction Cache

What it is: In-memory key-value store (e.g. Redis) of recent text→tags.

Why chosen: Many duplicate queries in chatbots or logs.

Trade-Offs

- **Benefit:** Reduces compute by up to 20 % under hit rates >30 %.
- **Drawback:** Staleness risk when models update; cache invalidation complexity.

Model Artifact Store

What it is: Object storage bucket for versioned model binaries.

Why chosen: Durable, low-cost storage with easy retrieval by inference nodes.

Trade-Offs

- **Benefit:** Easy rollback and A/B testing of multiple versions.
- **Drawback:** Cold start latency when updating; requires warm-up batching.

Metrics Database

What it is: Time-series or relational DB for SLA metrics.

Why chosen: Charts latency percentiles, error rates, capacity planning.

Trade-Offs

- **Benefit:** Alerting on anomalies; capacity forecasting.
- **Drawback:** Additional operational cost and complexity.

5. API Design

Endpoint	Method	Request	Response	Notes
<code>/ner/v1/predict</code>	POST	<code>{ text: string }</code>	<code>{ tokens: string[], tags: string[] }</code>	JSON over HTTPS; CORS enabled
<code>/ner/v1/models</code>	GET	—	<code>{ versions: [{ id, createdAt }] }</code>	List available model versions
<code>/ner/v1/metrics</code>	GET	<code>?from=&to=&metric=</code>	Time-series of latency or errors	Requires admin auth

- **Authentication:** JWT Bearer tokens with role-based claims
- **Rate limiting:** 100 RPS per API key, 429 on exceed
- **Versioning:** URI includes `v1`; future `v2` supported

6. Database Schema Design

Metadata DB (PostgreSQL)

sql

CopyEdit

```
CREATE TABLE models
  id          TEXT PRIMARY KEY,
  version     INT,
  path        TEXT,
  created_at  TIMESTAMP,
  active      BOOLEAN;
CREATE INDEX ON models(active);

CREATE TABLE request_logs
  id          UUID PRIMARY KEY,
  model_id    TEXT REFERENCES models(id),
  request_hash TEXT,
  latency_ms  INT,
  timestamp   TIMESTAMP;
CREATE INDEX ON request_logs(timestamp);
```

- **Why SQL:** Strong consistency for metadata; easy ad hoc queries.
- **Indexing:** `active` on models for fast lookup; `timestamp` on logs for range scans.
- **Cache (Redis):** Simple string→JSON mapping with TTL matching model TTL.

7. Summarize & Bottlenecks

- **Summary:**
This design meets 200 QPS peak with <200 ms latency, 99.9 % availability, and supports daily retraining and zero-downtime model rollouts.
- **Potential Bottlenecks & Mitigations:**
 1. **GPU Saturation:** monitor queue length; autoscale pods; fall back to CPU pods.
 2. **Cache Invalidation:** versioned cache keys; clear cache on model update.
 3. **Deployment Rollout Latency:** warm-up new nodes with sample inference.

4. **Data Surge:** burst buffer queue in SQS/Kafka to smooth traffic spikes.

Careful monitoring and autoscaling ensure the system remains performant as usage grows.